Interview_Prep - Combined Guide

DIRECTORY_INDEX.md

📁 Interview Prep Knowledge Base — Directory Index

> ***Last Updated:*** *2026-04-08* | ***Total Files:*** *55+* | ***Organized by Domain***

🗂️ New Organized Structure

```
Interview_Prep/
│
├── 📄 DIRECTORY_INDEX.md                    ← You are here
│
├── 🔴 SOC_Threat_Investigation/             (9 files | ~255 KB)
│   ├── Email_Security_SOC_Guide_Part1.md
│   ├── Email_Security_SOC_Guide_Part2.md
│   ├── Email_Security_SOC_Guide_Part3.md
│   ├── Threat_Hunting_SOC_Guide_Part1.md
│   ├── Threat_Hunting_SOC_Guide_Part2.md
│   ├── Threat_Hunting_SOC_Guide_Part3.md
│   ├── Threat_Hunting_SOC_Guide_Part4.md
│   ├── Reactive_SOC_Investigation_Guide_Part1.md
│   └── Reactive_SOC_Investigation_Guide_Part2.md
│
├── 🟠 Cisco_SOC_Prep/                      (4 files | ~74 KB)
│   ├── Cisco_SOC_Part1_Core_SOC_IR.md
│   ├── Cisco_SOC_Part2_ThreatIntel_MITRE_Hunting.md
│   ├── Cisco_SOC_Part3_Network_Endpoint_Tools.md
│   └── Cisco_SOC_Part4_Scenarios_Behavioral.md
│
├── 🟢 CNAPP_CSPM_Platforms/                 (2 files | ~105 KB)
│   ├── Wiz_CSPM_Interview_QA.md
│   └── Prisma_Cloud_CSPM_Interview_QA.md
│
├── 🟣 AWS_Security_QA/                      (5 files | ~118 KB)
│   ├── Answers_Section1_IAM.md
│   ├── Answers_Section2_Network_Section3_S3.md
│   ├── Answers_Section4_Encryption_Section5_Logging.md
│   ├── Answers_Section6_Compute_Section7_AppSec.md
│   └── Answers_Section8_Compliance_Section9_Grilling.md
│
├── 🟪 WellsFargo_Prep/                      (14 files | ~356 KB)
│   ├── WF_Senior_InfoSec_Mock_Interview_Guide.md
│   ├── WF_Senior_InfoSec_Part4_AppSec.md
│   ├── WF_Senior_InfoSec_Part5_CloudSec.md
│   ├── WF_Senior_InfoSec_Part6_GRC.md
│   ├── WF_Senior_InfoSec_Part7_IRSOC.md
│   ├── WF_Senior_InfoSec_Part8_Architecture.md
```

```
│   ├── WF_Interview_PowerBI_SQL_Excel.md
│   ├── WF_Interview_Pitch.md
│   ├── WF_VM_Self_Intro_Pitch.md
│   ├── WF_Gap_Notes_Part1_PowerBI_Excel.md
│   ├── WF_Gap_Notes_Part2_ServiceNow_SQL_Splunk.md
│   ├── WF_Gap_Notes_Part3_Azure_GCP_Wiz.md
│   ├── WF_Learning_Resources_Guide.md
│   └── WellsFargo_JD_Coverage_Analysis.md
│
├── 🟣 EY_Prep/                              (2 files | ~41 KB)
│   ├── EY_Cloud_Security_Interview_Prep.md
│   └── EY_CNAPP_Self_Intro.md
│
├── 🟤 General_Interview/                    (5 files | ~166 KB)
│   ├── Ultimate_Interview_Prep_Part1.md
│   ├── Ultimate_Interview_Prep_Part2.md
│   ├── Cloud_Security_HR_Interview_Top20.md
│   ├── Cloud_Security_Mock_Interview.md
│   └── Self_Intro.md
│
└── 🟤 Resume/                               (1 file | ~10 KB)
    └── GopiKrishna_Resume_HSBC_CloudSecurity.md
```

Also in the Parent CNAPP/ Directory:

```
CNAPP/
├── 🟡 Cloud_Security_Guides/               (12 files | ~519 KB + 7MB PDF)
│   ├── Advanced_Cloud_Security_Study_Guide.md
│   ├── Cloud_Security_Study_Guide.md
│   ├── Cloud_Security_Complete_Playbook.md
│   ├── Cloud_Security_Unified_Mastery_Guide.md   (205 KB — largest)
│   ├── Cloud_Security_Automation_Scripts.md
│   ├── cloud_security_interview_guide.md
│   ├── cloud_security_interview_guide.docx
│   ├── AWS_Security.pdf                          (7 MB)
│   ├── Financial_Compliance_Frameworks.md
│   ├── CNAPP_Structured_Guide.md
│   ├── CNAPP_Policy_Examples.md
│   └── KAC_and_Runtime_Detections_Guide.md
│
├── 🔵 Container_K8s_Security/              (4 files | ~142 KB)
│   ├── EKS_K8s_Security_CNAPP.md
│   ├── ECS_Container_Security_CNAPP.md
│   ├── K8s_Security_Manifests_Examples.md
│   └── container_mitre_scenarios.docx
│
├── Advanced_Cloud_Security_Study_Guide.md
├── INDEX.md
├── build_gitbook.py / index.html / pages_data.*
```

```
│
└── Interview_Prep/                          ← Organized above
```

📊 Category Summary

Color	Folder	Domain	Files	Purpose
🔴	`SOC_Threat_Investigation/`	SOC Ops, Threat Hunting, IR	9	Operational guides & playbooks
🟠	`Cisco_SOC_Prep/`	Cisco MSS SOC Role	4	Role-specific interview prep
🟢	`CNAPP_CSPM_Platforms/`	Wiz, Prisma Cloud	2	Platform-specific Q&A banks
🟣	`AWS_Security_QA/`	AWS Security Services	5	Service-by-service Q&A
🟣	`WellsFargo_Prep/`	Wells Fargo InfoSec	14	Company-specific prep
🟣	`EY_Prep/`	EY Cloud Security	2	Company-specific prep
🟤	`General_Interview/`	Cross-Company Prep	5	HR, behavioral, general Q&A
🟤	`Resume/`	Resume	1	Resume documents
🟡	`Cloud_Security_Guides/`	Cloud Security KB	12	Study guides (parent dir)
🔵	`Container_K8s_Security/`	Container Security	4	K8s/ECS/EKS guides (parent dir)

🔗 Reading Paths by Career Goal

🎯 SOC Analyst / Threat Hunter / IR Role

```
Reading Order:
1. SOC_Threat_Investigation/Email_Security_SOC_Guide_Part1-3
2. SOC_Threat_Investigation/Threat_Hunting_SOC_Guide_Part1-4
3. SOC_Threat_Investigation/Reactive_SOC_Investigation_Guide_Part1-2
4. Cisco_SOC_Prep/Parts 1-4
5. General_Interview/ (HR prep)
```

☁️ Cloud Security / CNAPP Engineer Role

```
Reading Order:
1. Cloud_Security_Guides/Cloud_Security_Unified_Mastery_Guide
2. Cloud_Security_Guides/Advanced_Cloud_Security_Study_Guide
3. CNAPP_CSPM_Platforms/Wiz + Prisma Cloud Q&A
4. AWS_Security_QA/Sections 1-9
5. Container_K8s_Security/ guides
6. General_Interview/ (HR prep)
```

🏦 Company-Specific Interview

```
Reading Order:
1. General_Interview/Self_Intro.md (customize)
2. General_Interview/Ultimate_Interview_Prep_Part1-2
3. General_Interview/Cloud_Security_HR_Interview_Top20
4. [Company Folder] — WellsFargo_Prep/ or EY_Prep/ or Cisco_SOC_Prep/
5. Resume/ — Review and tailor
```

Part1_Core_SOC_and_Resume.md

DevSecOps & Cloud Security Architect Interview Guide

SECTION 1 — Self Introduction & Resume Deep Dive

Q1: Walk me through your background and how your experience aligns with this Senior Security Analyst/Architect role.

What they are evaluating: Your ability to summarize 4 years of experience cohesively, highlighting relevant skills (Cloud Security, EDR, Incident Response) without getting bogged down in irrelevant details. They want to see communication skills and confidence.

Expert-Level Answer: "I have four years of dedicated experience in Security Operations, heavily focused on cloud security, threat hunting, and incident response. Currently, at UltraViolet Cyber, I act as a key player in our SOC, where I monitor and investigate complex alerts across hybrid environments using tools like CrowdStrike Falcon and SecureWorks Taegis XDR. My day-to-day involves deep-dive log analysis—correlating telemetry from AWS CloudTrail, GuardDuty, and EKS clusters with traditional endpoint logs to identify sophisticated threat actors. Recently, I've shifted significantly towards proactive security and DevSecOps. I manage Falcon CWPP deployments across AWS EC2 and Kubernetes (EKS) using DaemonSets, ensuring runtime protection. I also integrated Terraform code scanning into our CI/CD pipelines to catch insecure configurations before deployment, effectively shifting security left. My background started at Cisco, where I built a strong foundation in networking, firewall automation (ASA/FTD), and containerization. Ultimately, my transition from network engineering to cloud-native threat hunting allows me to not just detect threats, but architect secure, automated defenses against them."

Follow-up Grilling Questions:

- You mentioned managing Falcon CWPP on EKS. How exactly did you configure the DaemonSets, and how do you handle nodes that fail to deploy the sensor?
- How do you balance the noise of Shift-Left IaC scanning (Terraform) with developer velocity?

Common Mistakes Candidates Make:

- Reciting the resume bullet by bullet like a laundry list.
- Focusing too much on entry-level tasks (like basic SIEM monitoring) instead of architect-level achievements (like EKS DaemonSet deployments and CI/CD integrations).

Real-World Example: Instead of saying "I use CrowdStrike," emphasize: "When we deployed EKS, I identified a visibility gap. I authored the Kubernetes manifest to deploy Falcon as a DaemonSet to ensure every new worker node instantly spun up a sensor, guaranteeing zero runtime visibility gaps."

Q2: On your resume, you mention "Correlated logs from AWS CloudTrail, GuardDuty, Falcon telemetry... and NetFlow". Can you walk me through a specific investigation where you had to

correlate three or more of these sources?

What they are evaluating: Hands-on analytical methodology. Can you actually connect the dots between cloud control plane logs, endpoint execution, and network traffic, or are you just reading alerts off a single dashboard?

Expert-Level Answer: "Certainly. We had a GuardDuty alert trigger for anomalous IAM behavior—specifically, 'UnauthorizedAccess:IAMUser/InstanceCredentialExfiltration'.

1. **CloudTrail:** I immediately pivoted to CloudTrail and searched for the assumed role session. I identified that the `sts:AssumeRole` was called from an IP address outside our corporate VPN, and the actor was subsequently making `ec2:DescribeInstances` and `s3:ListBuckets` API calls.
2. **Falcon Telemetry:** I took the instance ID that originally owned that IAM role and queried CrowdStrike Falcon. I found a suspicious `curl` command hitting the AWS metadata service (`169.254.169.254/latest/meta-data/iam/security-credentials/`) originating from a Python script running under a web server daemon.
3. **NetFlow/WAF Logs:** To determine how the web server was compromised, I correlated the timestamp of the payload drop with our WAF and NetFlow logs, identifying an initial Server-Side Request Forgery (SSRF) payload successfully bypassing our WAF rules. By correlating these three, we identified the entire kill chain: SSRF -> Metadata Exfiltration -> External API enumeration, and isolated the EC2 instance immediately while rotating the IAM credentials."

Follow-up Grilling Questions:

- In that scenario, how fast does GuardDuty generate that alert? Is there a delay? (Hint: GuardDuty can have a 15-20 minute delay).
- How would you automate the containment of that exact attack path?

Common Mistakes Candidates Make:

- Giving a theoretical answer instead of a step-by-step technical narrative.
- Failing to mention the exact logs or API calls (e.g., just saying "I checked AWS" instead of "I queried CloudTrail for `sts:AssumeRole` ").

Real-World Example: This exact scenario mimics the Capital One breach methodology (SSRF to Metadata service to S3 exfiltration). Demonstrating you know how to trace this specific path is highly impressive.

SECTION 2 — SOC Operations

Q3: How do you differentiate a True Positive from a False Positive when an EDR triggers an alert for "Suspicious PowerShell Execution"?

What they are evaluating: Your analytical process and understanding of LOLBins (Living Off the Land Binaries). Do you blindly trust alerts, or do you analyze the command line arguments and process lineage?

Expert-Level Answer: "A 'Suspicious PowerShell Execution' alert requires immediate context gathering. To determine if it's a True Positive, I look at the **Process Lineage** and the **Command Line Arguments**. First, I check the parent process. If `powershell.exe` was spawned by `winword.exe` (Microsoft Word) or `wsmprovhost.exe` (WinRM), that is highly anomalous and leans towards a True Positive—likely a macro or lateral movement. If it was spawned by `explorer.exe` or `sccm.exe` (System Center), it requires further digging. Second, I analyze the arguments. I look for obfuscation (e.g., mixed case, backticks), encoded commands (`-enc` , `-EncodedCommand`), execution policy bypasses (`-ep bypass`), or window hiding (`-w hidden`). Third, I look at network connections originating from that specific PID. Is it reaching out to a raw IP address over port 443, or a known malicious domain? If the script is a known IT admin script running from a centralized share with standard arguments, I classify it as a False Positive and tune the detection rule to exclude that specific hash or file path to reduce SOC fatigue."

Follow-up Grilling Questions:

- What if the PowerShell script is running purely in memory (fileless)? How does CrowdStrike Falcon see it? (Hint: AMSI integration / Script Control).
- If it is a True Positive and actively downloading a payload, what is your immediate next step?

Common Mistakes Candidates Make:

- Just saying "I check VirusTotal." (PowerShell is a legitimate tool; VT won't flag the `powershell.exe` binary).
- Not mentioning parent-child process relationships.

Real-World Example: Identifying that a developer legitimately uses `-ep bypass` for a build script, and creating an IOA (Indicator of Attack) exclusion in Falcon specifically for that developer's machine and script path, rather than globally whitelisting the command.

Q4: You notice a sudden spike in MTTD (Mean Time to Detect) and MTTR (Mean Time to Respond) in the SOC. As a senior analyst, how do you address this?

What they are evaluating: SOC maturity, leadership, and process improvement skills. Can you think like a SOC Manager?

Expert-Level Answer: "A spike in MTTD and MTTR usually indicates either an influx of noisy alerts (alert fatigue), a lack of clear playbooks, or a tooling failure. I would take a data-driven approach to fix this:

1. **Analyze the Top Talkers:** I'd pull a report from Taegis XDR or Splunk to identify which rules are firing the most. Often, 80% of the noise comes from 20% of the rules.
2. **Detection Tuning:** For high-volume false positives, I would refine the logic—adding exclusions for known benign behavior or correlating it with secondary indicators before triggering a high-severity alert.
3. **SOAR Automation:** If the alerts are True Positives but routine (e.g., phishing emails or impossible travel), I would leverage SOAR (like Shuffle, which I've used) to automate the initial triage. For example, automatically extracting URLs, querying MISP/VirusTotal, and disabling the user account if malicious.
4. **Playbook Refinement:** I would review our SOPs. If analysts don't know exactly what to do when a specific alert fires, MTTR skyrockets. I'd ensure the playbook is explicitly linked in the alert notes."

Follow-up Grilling Questions:

- How do you convince management to dedicate time to tuning when the queue is overflowing with active alerts?
- Describe a time you automated a task that significantly reduced MTTR.

Common Mistakes Candidates Make:

- Blaming junior analysts for being slow.
- Throwing more headcount at the problem instead of tuning and automation.

SECTION 16 — Mock HR Round

Q5: Tell me about a time you had a conflict with a developer or an infrastructure team regarding a security implementation. How did you resolve it?

What they are evaluating: Empathy, communication, and business acumen. Security is often seen as a blocker; they want to see if you are a business enabler.

Expert-Level Answer: "During my time at UltraViolet, we were rolling out CrowdStrike Falcon CWPP across our Amazon EKS clusters. The DevOps team pushed back heavily, concerned that the DaemonSet would consume too

many node resources and impact application performance. Instead of forcing the mandate, I sat down with their lead engineer. I agreed to a phased rollout. We deployed the sensor to a non-production staging cluster first. I set up Datadog dashboards to monitor CPU and memory consumption of the Falcon pods specifically. After a week, we reviewed the data together, which showed the sensor utilized less than 1% of CPU and minimal memory. By providing empirical data and treating them as partners rather than adversaries, they became comfortable with the rollout, and we successfully deployed it to production without further friction."

Follow-up Grilling Questions:

- What if the sensor *did* cause a CPU spike? What would have been your compromise?

Q6: Why are you looking to leave your current role at UltraViolet Cyber?

What they are evaluating: Professionalism and career trajectory.

Expert-Level Answer: "I've had a great experience at UltraViolet Cyber, growing from fundamental SOC monitoring to leading complex cloud investigations and Kubernetes security deployments. However, I am now looking for a role that leans heavier into Cloud Security Architecture and Detection Engineering. I want to build defenses and DevSecOps pipelines at a larger scale, and this organization's focus on mature cloud-native infrastructure aligns perfectly with where I want to take my career next."

SECTION 17 — Final Rapid Fire Round

Q: Port 3389 is open to the internet on an EC2 instance. What is the immediate risk, and what is the AWS remediation? A: RDP brute force or BlueKeep exploitation. Remediation: Modify the attached Security Group to remove the 0.0.0.0/0 inbound rule for 3389 and restrict it to a specific corporate VPN IP or use AWS Systems Manager (SSM) Fleet Manager instead of exposing RDP.

Q: What is the difference between a Bind Shell and a Reverse Shell? A: In a Bind Shell, the attacker connects to a port opened by the victim machine. In a Reverse Shell, the victim machine actively calls back out to the attacker's listening machine (often bypassing inbound firewall rules).

Q: You see `svchost.exe` running from `C:\Users\Public`. What is your conclusion? A: 100% malicious. `svchost.exe` should strictly execute from `C:\Windows\System32`. It is likely malware masquerading as a legitimate system process.

Q: What HTTP status code indicates an SSRF attempt might have been successful in hitting the AWS Metadata service? A: HTTP 200 OK.

Q: How do you grep for an IP address in a log file? A: `grep -E -o "([0-9]{1,3}[\.]){3}[0-9]{1,3}" /var/log/syslog`

Q: What is the primary purpose of an AWS IAM SCP (Service Control Policy)? A: It acts as a guardrail at the AWS Organization level, defining the maximum available permissions for member accounts, regardless of what local IAM policies allow.

Part2_Cloud_and_Container_Security.md

DevSecOps & Cloud Security Architect Interview Guide

SECTION 4 — AWS Cloud Security

Q1: You receive an alert from GuardDuty for `Recon:EC2/PortProbeUnprotectedPort`. Upon investigation, you see an EC2 instance has port 22 open to `0.0.0.0/0`. Walk me through how you investigate and remediate this from start to finish.

What they are evaluating: Incident response methodology in the cloud. They want to see if you immediately terminate the instance (bad) or if you isolate it and check for lateral movement first (good).

Expert-Level Answer: "First, I would validate the alert. I'd check the Security Group associated with the EC2 instance via AWS CLI or Console to confirm port 22 is indeed open to the internet. Second, I wouldn't immediately terminate the instance. Instead, I would **contain** it. I'd change the Security Group to a strict isolation group that blocks all inbound/outbound traffic except for our forensic/IR tools (like CrowdStrike or SSM). Third, I'd investigate for compromise. I'd check CloudTrail to see who modified the Security Group recently (`AuthorizeSecurityGroupIngress`). I'd also query Falcon EDR telemetry to see if there were successful SSH logins (`Event: UserLogon`) around the time of the port probe, and check for any anomalous child processes spawned by `sshd`. If compromised, I'd trigger the IR playbook: snapshot the EBS volume for forensics, tag the instance as 'Compromised', and coordinate with the asset owner to rebuild the server from a clean AMI. Finally, I'd implement a preventative control—such as an AWS Config Rule or Terraform check—to automatically flag or revert Security Groups opening port 22 globally."

Follow-up Grilling Questions:

- What if the instance is part of an Auto Scaling Group? If you isolate it, won't the ASG just spin up a new vulnerable instance?
- How do you find the exact IAM user who opened the port in CloudTrail?

Common Mistakes Candidates Make:

- Saying "I'll just delete the instance." (Destroys forensic evidence).
- Focusing only on the AWS console and forgetting to check the endpoint (EDR) for actual compromise.

Real-World Example: In my SOC environment, developers sometimes temporarily opened SSH for debugging and forgot to close it. We moved away from SSH entirely by implementing AWS Systems Manager (SSM) Session Manager, which doesn't require open inbound ports.

Q2: An attacker compromises an EC2 instance that has an overly permissive IAM role attached. Explain the exact mechanism of how they extract the credentials and what they could do with them.

What they are evaluating: Understanding of the Instance Metadata Service (IMDS) and Server-Side Request Forgery (SSRF) to IAM abuse vectors.

Expert-Level Answer: "Once an attacker gains remote code execution on the EC2 instance, they can query the Instance Metadata Service (IMDS) using a simple HTTP request to a non-routable IP. They would execute a command like `curl http://169.254.169.254/latest/meta-data/iam/security-credentials/<RoleName>`. The response contains an `AccessKeyId`, a `SecretAccessKey`, and a `Token`. The attacker can copy these credentials and configure them on their own local machine using `aws configure`. Once configured locally, the attacker assumes the identity of that EC2 instance. If the role has `s3:GetObject` and `s3:ListBucket`, they can exfiltrate data to their own machine. If it has `iam:CreateUser` or `iam:AttachUserPolicy`, they can create a backdoor admin account for persistence."

Follow-up Grilling Questions:

- How does AWS IMDSv2 mitigate this specific attack?

- If you see those temporary credentials being used from an external IP address in CloudTrail, what is the fastest way to revoke them?

Common Mistakes Candidates Make:

- Confusing EC2 Instance Profiles with IAM Users. (Instance profiles generate temporary STS tokens, not permanent access keys).
- Not knowing the IP address `169.254.169.254`.

SECTION 5 — Kubernetes & EKS Security

Q3: You mentioned deploying Falcon CWPP as a DaemonSet in EKS. Why a DaemonSet, and how does CrowdStrike gain visibility into other containers running on that node?

What they are evaluating: Deep understanding of Kubernetes architecture and how container security sensors actually function at the OS level.

Expert-Level Answer: "We deploy it as a DaemonSet because Kubernetes guarantees that a DaemonSet ensures exactly one copy of the pod runs on every single worker node in the cluster. This is crucial for security because as the EKS cluster scales up and adds new nodes, the Falcon sensor is automatically provisioned without manual intervention, ensuring zero coverage gaps. CrowdStrike gains visibility into other containers because the Falcon pod runs with elevated privileges on the host—specifically using `hostPID` and `hostNetwork`, and it mounts the host's `/var/run/docker.sock` or `containerd.sock`. Because containers are essentially just isolated processes sharing the same host kernel, the Falcon sensor hooks into the host's kernel (often using eBPF) to monitor syscalls across all container namespaces. This allows it to see exactly what processes are executing inside every other pod."

Follow-up Grilling Questions:

- Running a security pod with high privileges is inherently risky. How do you secure the Falcon DaemonSet itself?
- If a developer deploys a pod with `privileged: true`, how does that bypass standard namespace isolation?

Common Mistakes Candidates Make:

- Not understanding *why* a DaemonSet is used over a Deployment.
- Thinking the Falcon sensor is injected *into* every other container, rather than running alongside them and monitoring the shared kernel.

Real-World Example: This is the standard architectural deployment for almost all CWPPs (CrowdStrike, Aqua, Prisma Cloud). When I deployed this at UltraViolet, I had to ensure our OPA Gatekeeper policies allowed the CrowdStrike namespace to bypass our strict "No Privileged Pods" rule.

Q4: An attacker compromises a web application pod running in EKS. What are the common techniques they would use to breakout of the container or escalate privileges within the cluster?

What they are evaluating: Kubernetes threat modeling and MITRE ATT&CK for Containers.

Expert-Level Answer: "If a pod is compromised, the attacker's first goal is usually discovery and lateral movement.

1. **Service Account Token Abuse:** Every pod mounts a default service account token at `/var/run/secrets/kubernetes.io/serviceaccount/token`. The attacker will grab this token and

attempt to query the Kubernetes API server. If RBAC is misconfigured (e.g., the service account has `cluster-admin` or can list secrets), they can dump all cluster secrets.
2. **Container Breakout:** If the pod was deployed with `securityContext: privileged: true`, the attacker has almost root-level access to the underlying worker node. They can execute `chroot /host` to escape the container boundary and take over the underlying EC2 node.
3. **Cloud Metadata Abuse:** If IMDSv2 isn't enforced, or if the pod isn't restricted by network policies, they can hit `169.254.169.254` to steal the worker node's underlying AWS IAM credentials."

Follow-up Grilling Questions:

- How do you detect someone querying the Kubernetes API server anomalously? (Hint: Kubernetes Audit Logs).
- How would you use IAM Roles for Service Accounts (IRSA) to mitigate the cloud metadata abuse?

SECTION 13 — Architecture & Design Questions

Q5: We are migrating a monolithic application to a microservices architecture on AWS EKS. As a Security Architect, design the security controls you would implement across the entire lifecycle (Code to Cloud).

What they are evaluating: Holistic DevSecOps and Shift-Left thinking. Can you design a secure pipeline rather than just reacting to alerts?

Expert-Level Answer: "I would architect security in three distinct phases: Build, Deploy, and Run. **1. Build Phase (Shift-Left):**

- I'd integrate SAST tools (like SonarQube) into the Git repository to catch vulnerable code on Pull Requests.
- I'd implement SCA (Software Composition Analysis) like OWASP Dependency-Check or Snyk to catch vulnerable open-source libraries.
- I'd integrate a container scanner (like Trivy) into the CI pipeline to scan the Docker image for vulnerabilities *before* it's pushed to the Elastic Container Registry (ECR).

2. Deploy Phase (Infrastructure as Code):

- Since we use Terraform, I'd integrate `tfsec` or `checkov` to scan the IaC for misconfigurations (e.g., ensuring S3 buckets aren't public, or EKS endpoints are private).
- Within Kubernetes, I'd implement an Admission Controller like OPA Gatekeeper or Kyverno. If a developer tries to deploy an image that hasn't been scanned or tries to run a pod as `root`, the Admission Controller rejects the deployment.

3. Run Phase (Runtime Protection):

- I'd deploy CrowdStrike Falcon CWPP as a DaemonSet on the EKS nodes for kernel-level visibility and threat detection.
- I'd implement Kubernetes Network Policies to enforce default-deny traffic between microservices, so if the frontend is compromised, it can't natively talk to the backend database.
- Finally, I'd feed CloudTrail, EKS Audit Logs, and Falcon telemetry into our SIEM (Taegis XDR/Splunk) for continuous SOC monitoring."

Follow-up Grilling Questions:

- Developers complain that the Trivy scanner is breaking the build due to unpatchable 'High' vulnerabilities in base images. How do you handle this?

- How do you handle secrets management in this architecture? Do you store them in Kubernetes Secrets or something external?

Common Mistakes Candidates Make:

- Only talking about runtime security (EDR/Firewalls) and ignoring the CI/CD pipeline.
- Not mentioning Admission Controllers, which are the backbone of Kubernetes security enforcement.

Part3_EDR_SIEM_and_Log_Analysis.md

DevSecOps & Cloud Security Architect Interview Guide

SECTION 3 — CrowdStrike Falcon Deep Technical

Q1: An alert fires in CrowdStrike Falcon for a "High Severity: OverWatch detection". You check the process tree, and it's simply `cmd.exe` running `ping 8.8.8.8`. Why did Falcon flag this, and how do you investigate?

What they are evaluating: Understanding of behavioral heuristics, IOAs (Indicators of Attack), and process lineage vs. just looking at the binary.

Expert-Level Answer: "Falcon doesn't just alert on bad hashes; it alerts on anomalous behavior (IOAs). If `ping 8.8.8.8` triggers an OverWatch (threat hunting) detection, I immediately look at the **Process Lineage**. If the parent process is `winword.exe` (Microsoft Word), `excel.exe`, or `w3wp.exe` (IIS web server), that is a massive red flag. `winword.exe` should never spawn a command prompt to ping an external IP. This usually indicates a malicious macro payload checking for internet connectivity before downloading the second-stage payload. To investigate:

1. I would expand the process tree in the Falcon UI to see the exact parent and grandparent processes.
2. I would check the 'Network Operations' tab for that PID to see if the macro subsequently reached out to a suspicious domain.
3. I would network-contain the host via Falcon immediately to prevent the second-stage download or lateral movement, then retrieve the malicious Word document for sandbox analysis."

Follow-up Grilling Questions:

- What if the parent process is `explorer.exe`?
- How do you pull a file from an endpoint remotely using CrowdStrike? (Hint: Real Time Response / RTR).

Common Mistakes Candidates Make:

- Dismissing it as a False Positive simply because `ping.exe` is a legitimate Windows binary.
- Not understanding what CrowdStrike OverWatch actually is (human-led threat hunting).

Real-World Example: This exact pattern is used by Emotet and Trickbot. The macro runs `ping` with a delay to evade sandbox detection before reaching out to the C2 server.

Q2: You need to investigate a machine that you suspect is compromised, but it's currently isolated via CrowdStrike Network Containment. How do you investigate it, and what commands would you run?

What they are evaluating: Knowledge of CrowdStrike's Real Time Response (RTR) capabilities and live forensics.

Expert-Level Answer: "When a machine is Network Contained in Falcon, it drops all network connections except the persistent TLS connection to the CrowdStrike cloud. I would use **Real Time Response (RTR)** to establish a remote shell into the isolated host. Once connected, I would execute several live response commands:

1. `ps` - to list running processes and look for anomalies not caught by the sensor.
2. `netstat` - to check for active or listening ports (though external connections will be blocked, local bind shells might be visible).
3. `cd` and `ls` - to navigate to suspicious directories like `C:\Users\Public` or `%TEMP%`.
4. `get` - to pull a suspicious file or memory dump off the machine and upload it to the Falcon cloud for my review.
5. If I need to run a custom PowerShell script to hunt for specific IOCs, I would use the `runscript` command to execute a pre-approved script from our Falcon repository."

Follow-up Grilling Questions:

- What permissions do you need in Falcon to use the `runscript` or `get` commands? (Hint: RTR Active Responder / RTR Admin).
- If the attacker achieves SYSTEM privileges and uninstalls the Falcon sensor, what happens? (Hint: Sensor Tampering Protection).

SECTION 10 — SIEM/XDR & Log Correlation

Q3: You have logs coming into Splunk/Taegis XDR from AWS CloudTrail, CrowdStrike, and Cisco FTD Firewalls. How would you correlate these logs to track an attacker who compromised an EC2 instance and exfiltrated data?

What they are evaluating: Understanding of log schemas, correlation keys, and SIEM search logic.

Expert-Level Answer: "To track the full kill chain, I need to pivot between log sources using common correlation keys—primarily IP addresses, timestamps, and hostnames/instance IDs.

1. **Initial Access (Cisco FTD):** I'd query the firewall logs filtering by the EC2 instance's public IP. I'd look for anomalous inbound traffic, such as SSH brute force or an HTTP exploit attempt. The correlation key here is the **Destination IP** (EC2 public IP) and **Source IP** (Attacker).
2. **Execution (CrowdStrike):** Using the timestamp from the firewall log, I'd query Falcon logs (or use the Falcon console) for that specific EC2 instance's hostname. I'd look for process executions (e.g., `wget`, `curl`, `bash -i`) originating from the web server daemon. Correlation key: **Hostname / Local IP**.
3. **Privilege Escalation / Cloud Abuse (AWS CloudTrail):** If the attacker stole the IAM role from the instance metadata, I would take the IAM Role ARN found on that EC2 instance and query CloudTrail. My query would look for `userIdentity.arn` matching the role, but where the `sourceIPAddress` does *not* match our VPC NAT Gateway or corporate IPs. This reveals what AWS API calls the attacker made externally.
4. **Exfiltration (Cisco FTD / CloudTrail):** I'd check CloudTrail for `s3:GetObject` if they stole data from S3, or check the firewall/VPC Flow Logs for massive outbound bytes (e.g., 50GB transferred out) from the EC2 instance to the attacker's IP."

Follow-up Grilling Questions:

- How do you handle timestamp discrepancies between AWS (UTC), Firewalls (Local), and endpoints?
- In Splunk, how would you write a `stats` or `transaction` command to link these together?

Common Mistakes Candidates Make:

- Giving vague answers like "I'll just search for the IP in Splunk." You must specify the fields and the logic.

- Forgetting that CloudTrail logs external API usage, which is the most critical part of an AWS breach.

SECTION 11 — Networking & Packet Analysis

Q4: You capture a PCAP of suspicious traffic. You see a DNS request for a very long, random string like `jh234g23j4hg234.maliciousdomain.com`. What is happening here, and how do you investigate?

What they are evaluating: Deep networking knowledge and understanding of DNS Data Exfiltration / C2.

Expert-Level Answer: "This is highly indicative of **DNS Tunneling** or **DNS Data Exfiltration**. Because DNS is rarely blocked outbound by corporate firewalls, attackers use it to bypass restrictions. The attacker encodes stolen data (like passwords or sensitive files) into Base64 or Hex, appends it as a subdomain to a domain they control (`maliciousdomain.com`), and makes a DNS TXT or A record request. The corporate DNS server recursively forwards this to the attacker's authoritative name server, effectively delivering the stolen data. To investigate:

1. In Wireshark, I would filter by `dns` and look at the query lengths. A high volume of unique, exceptionally long subdomains to a single domain is a dead giveaway.
2. I would check the response size. If it's a Command and Control (C2) channel, the attacker's server will respond with TXT records containing commands to execute.
3. To remediate, I would immediately block `maliciousdomain.com` on our DNS sinkhole (like Cisco Umbrella or Pi-hole) and our perimeter firewalls. Then, I'd trace the source IP of the DNS request back to the endpoint and isolate it using CrowdStrike."

Follow-up Grilling Questions:

- How is this different from Domain Generation Algorithms (DGA)?
- If the traffic is encrypted using DoH (DNS over HTTPS), how can you detect it?

Common Mistakes Candidates Make:

- Confusing DNS tunneling with DGA (DGA is used by malware to find its C2 server by generating thousands of domains; Tunneling is using the DNS protocol itself to transmit data).

Real-World Example: Tools like `Iodine` or `Dnscat2` are specifically designed to create these tunnels. In a SOC environment, you should have SIEM alerts configured to trigger when the average length of DNS queries from a single host exceeds a specific threshold.

Part4_IR_Hunting_and_Malware.md

DevSecOps & Cloud Security Architect Interview Guide

SECTION 6 — Threat Hunting & Detection Engineering

Q1: You have 4 hours of free time this week to conduct a proactive threat hunt. You don't have any specific IOCs. Walk me through your methodology.

What they are evaluating: Do you understand hypothesis-driven threat hunting, or do you just search for bad hashes on VirusTotal?

Expert-Level Answer: "Without specific IOCs, I perform **Hypothesis-Driven Threat Hunting** mapped to the MITRE ATT&CK framework.

1. **Formulate a Hypothesis:** My hypothesis might be: 'Attackers are bypassing our email filters and using LOLBins (Living off the Land Binaries) for execution.'
2. **Identify the Data Source:** I'd use CrowdStrike Falcon and Taegis XDR telemetry, specifically focusing on Process Execution logs (EID 4688).
3. **Execute the Hunt:** I would write a Splunk query to look for instances of `certutil.exe`, `bitsadmin.exe`, or `powershell.exe` making outbound external network connections. Specifically, I'd filter out known IT subnets and look for `certutil -urlcache -split -f`.
4. **Analyze the Results:** If I get 10,000 hits, the hunt is too broad. I'd stack the data (frequency analysis) to find the outliers—for instance, 9,990 hits go to a known Microsoft update server, but 10 hits go to a raw IP address. I focus on those 10.
5. **Actionable Output (Detection Engineering):** If I find evil, I initiate Incident Response. If I find nothing, but the query was high-fidelity, I convert that Splunk query into a permanent Detection Rule so the SOC is alerted automatically next time. This is the core of Detection Engineering."

Follow-up Grilling Questions:

- What is "Stacking" or "Frequency Analysis" in threat hunting?
- How do you measure the success of a threat hunt if you don't find any attackers?

Common Mistakes Candidates Make:

- Saying "I'll search for IOCs from Twitter." (That is reactive, not proactive hunting).
- Forgetting the final step (converting successful hunts into automated detections).

SECTION 7 — Incident Response Scenarios

Q2: A user reports clicking a phishing link and entering their Office 365 credentials. Two hours later, you get an alert. Walk me through your entire Incident Response Lifecycle for this event.

What they are evaluating: Your adherence to structured IR frameworks (NIST SP 800-61 / SANS PICERL).

Expert-Level Answer: "I follow the SANS Incident Response lifecycle: Preparation, Identification, Containment, Eradication, Recovery, and Lessons Learned.

1. **Identification:** I would query our Azure AD / Entra ID sign-in logs for the user's account. I'm looking for 'Successful Logins' from anomalous IP addresses, impossible travel, or unrecognized devices over the last two hours.
2. **Containment:** If I confirm unauthorized access, I immediately contain the threat by:
    - Forcing a password reset and revoking all active sessions in Office 365.
    - Network containing the user's laptop via CrowdStrike in case the phishing link also dropped malware.
3. **Investigation/Eradication:** I would check O365 Audit Logs to see what the attacker did post-compromise. Did they create Inbox Rules to forward emails externally? Did they download sensitive files from SharePoint? Did they send internal phishing emails to other employees? I will delete any malicious inbox rules and trace any lateral phishing.
4. **Recovery:** Once clean, I restore the user's access, un-isolate their machine (if clean), and monitor their account closely for 48 hours.
5. **Lessons Learned:** I would extract the original phishing email via Proofpoint, extract the URL, block it on our web proxy (Zscaler), and recommend targeted security awareness training for that user."

Follow-up Grilling Questions:

- What if the attacker bypassed MFA? How is that possible? (Hint: Adversary-in-the-Middle (AiTM) attacks using proxy tools like Evilginx2).
- How do you detect malicious Inbox Rules using PowerShell or SIEM?

Common Mistakes Candidates Make:

- Forgetting to check for Inbox Rules (attackers almost always set rules to hide their activity).
- Not revoking active sessions (just changing the password doesn't kick out an attacker who already has a valid session token).

SECTION 8 — Ransomware & Malware Investigations

Q3: You get an alert from Falcon that Ransomware was blocked on a File Server. The business owner says, "CrowdStrike blocked it, we are safe, let's move on." Do you agree? What do you do next?

What they are evaluating: Understanding of the ransomware lifecycle. Ransomware execution is the *last* step of an attack, not the first.

Expert-Level Answer: "Absolutely not. Ransomware execution is the final payload of a breach. If CrowdStrike blocked the encryption attempt, it means the attacker has likely been in our network for days or weeks. My immediate thought is: **How did they get there, and what did they steal?** Modern ransomware actors operate on double extortion—they steal data first, then encrypt.

1. I would keep the File Server network-isolated.
2. I would trace the process lineage in Falcon. If the ransomware was executed via `psexec` or WMI, it means the attacker has lateral movement capabilities and likely compromised a Domain Admin account.
3. I would hunt for the initial entry vector—was it an unpatched VPN appliance? A phishing email? An exposed RDP port?
4. I would look for exfiltration indicators. Did we see massive outbound traffic on the firewall? Did they install tools like Rclone or MegaSync? Until I find Patient Zero, identify the lateral movement path, and reset all compromised credentials, the network is still heavily compromised, and they will simply try to deploy the ransomware again."

Follow-up Grilling Questions:

- If the attacker used `psexec`, what Windows Event IDs would you look for on the domain controller? (Hint: Event ID 4624 Logon Type 3, Event ID 7045 Service Creation).
- How do you handle evidence preservation if you need to wipe the machine?

Common Mistakes Candidates Make:

- Agreeing with the business owner and closing the ticket.
- Failing to mention data exfiltration (double extortion).

Real-World Example: In the Conti and LockBit playbook, actors spend weeks enumerating active directory and exfiltrating data before dropping the encryptor. Catching the encryptor means you missed the entire reconnaissance and exfiltration phase.

Part5_VM_Automation_and_DevSecOps.md

DevSecOps & Cloud Security Architect Interview Guide

SECTION 9 — Vulnerability Management

Q1: Nessus reports over 10,000 "High" and "Critical" vulnerabilities across our AWS infrastructure. As the Security Lead, how do you prioritize remediation without overwhelming the engineering teams?

What they are evaluating: Risk-based Vulnerability Management (RBVM) and process maturity. Do you just throw a 500-page PDF at developers, or do you curate actionable intelligence?

Expert-Level Answer: "You cannot patch 10,000 vulnerabilities overnight, so prioritization must be entirely risk-based. I do not rely solely on CVSS scores, as a CVSS 9.8 on an internal, air-gapped test server is less critical than a CVSS 7.5 on a public-facing web server.

1. **Asset Criticality & Exposure:** I prioritize vulnerabilities on internet-facing assets (EC2 instances with public IPs, ALBs) and critical business databases first.
2. **Threat Intelligence / EPSS:** I cross-reference the CVEs with Threat Intelligence (like CrowdStrike Falcon Spotlight) or CISA's KEV (Known Exploited Vulnerabilities) catalog. If a vulnerability is actively being exploited in the wild, it jumps to the front of the line.
3. **Compensating Controls:** If a server has a vulnerable Apache version, but it sits behind a WAF that blocks the specific exploit payload, the priority drops, buying us time to patch during the normal cycle.
4. **Automation & Jira:** Finally, I automate the workflow. I use the Nessus API or a Python script to group similar vulnerabilities (e.g., 'Update OpenSSL on 50 hosts') and create a single Jira epic for the infrastructure team, complete with exact remediation steps, rather than opening 50 individual tickets."

Follow-up Grilling Questions:

- How do you handle 'Zero-Day' vulnerabilities where no patch exists yet (e.g., Log4Shell on day 1)?
- Developers say they can't patch an out-of-date Java application because it will break legacy code. What is your response?

Common Mistakes Candidates Make:

- Relying strictly on CVSS scores.
- Not grouping tickets in Jira, which leads to ticket fatigue and developer pushback.

SECTION 14 — Automation & Scripting

Q2: You mentioned automating firewall tasks with Python and using Shuffle SOAR. Can you walk me through a specific script or playbook you built from scratch that saved your team significant time?

What they are evaluating: Actual coding/scripting experience vs. just running pre-built tools.

Expert-Level Answer: "At Cisco, I worked on a Python script to automate Firewall configurations. Managing ACLs across hundreds of ASA and FTD firewalls manually was error-prone. I utilized the `Netmiko` library. I wrote a script that would parse a CSV file containing required source IPs, destination IPs, and ports. The script would iterate through the CSV, SSH into the target firewall, and push the configuration commands dynamically. To ensure safety, I implemented a 'dry-run' feature that used the firewall's specific syntax checker before committing, and automatically generated a rollback configuration file in case the new ACL broke connectivity. In my SOC role, I utilized Shuffle SOAR to automate phishing triage. I built a playbook triggered by a webhook from Proofpoint. The playbook extracted URLs and file hashes from the email, sent them to VirusTotal and URLScan.io APIs for reputation checking, and if the score was above a malicious threshold, it automatically created an alert in Taegis XDR and updated the status to 'High Confidence', saving analysts about 15 minutes per phishing email."

Follow-up Grilling Questions:

- In your Python script, how did you handle credentials securely? Did you hardcode them? (Hint: Environment variables, AWS Secrets Manager, or HashiCorp Vault).
- How do you handle API rate limits when your SOAR playbook queries VirusTotal?

Common Mistakes Candidates Make:

- Describing a script but being unable to name the libraries used (e.g., Netmiko, Paramiko, Requests, Boto3).
- Admitting to hardcoding passwords in scripts.

SECTION 15 — DevSecOps & Shift-Left Security

Q3: A developer pushes a Terraform configuration that creates an S3 bucket with `acl = "public-read"`. How do you architect a DevSecOps pipeline to prevent this from reaching production?

What they are evaluating: Practical knowledge of CI/CD pipelines, IaC scanning, and enforcement mechanisms.

Expert-Level Answer: "To prevent insecure Infrastructure as Code (IaC) from reaching production, I would implement **Shift-Left Security** using a tool like `tfsec` or `checkov`.

1. **Pre-Commit Hook:** Ideally, developers have a pre-commit hook installed locally that runs `checkov` on their Terraform code. This gives them instant feedback before they even commit the code.
2. **CI Pipeline Integration:** Once they push the code to GitHub/GitLab and create a Pull Request, a CI action is triggered. The runner executes `checkov -d .` against the repository.
3. **Enforcement/Blocking:** Because `acl = "public-read"` violates a critical security policy, the CI pipeline is configured to fail the build. The Pull Request cannot be merged into the `main` branch until the developer changes the ACL to `private` or removes the block.
4. **Cloud Security Posture Management (CSPM):** As a fail-safe, if someone creates a public bucket manually via the AWS Console (bypassing Terraform), our CrowdStrike CSPM or AWS Config will detect it post-deployment and can trigger an automated Lambda function to revert the bucket to private."

Follow-up Grilling Questions:

- What if the developer absolutely *needs* the bucket to be public for a static website? How do you create an exception in the IaC scanner?
- How is `tfsec` different from a DAST (Dynamic Application Security Testing) tool?

Common Mistakes Candidates Make:

- Only talking about post-deployment detection (CSPM) instead of pre-deployment prevention (IaC scanning).
- Not understanding how CI/CD blocking mechanisms actually work (failing the exit code of the pipeline job).

SECTION 12 — Behavioral & Situation-Based Questions

Q4: Tell me about a time you made a significant mistake at work. How did you handle it?

What they are evaluating: Accountability, transparency, and the ability to learn from failure without deflecting blame.

Expert-Level Answer: "Early in my career at Cisco, I was tasked with updating an ACL on an ASA firewall using my Python automation script. I accidentally applied a broad 'deny ip any any' rule to the wrong interface during a maintenance window, effectively dropping connectivity for a subnet of users. I realized it immediately when my SSH

session hung. Instead of hiding it, I immediately jumped on the incident bridge, owned the mistake, and stated exactly what happened. Because I had built a rollback configuration feature into my script, I was able to log in via an out-of-band management console and revert the change within 5 minutes. After the incident, I didn't just apologize; I updated the Python script to include a secondary validation check that prompts the user to manually confirm the target interface name before executing any disruptive commands. It taught me that owning your mistakes immediately builds trust, and fixing the underlying process is more important than just fixing the immediate outage."

Follow-up Grilling Questions:

- Have you ever disagreed with a manager's technical decision? How did you handle it?

Real-World Example: This is the classic "I brought down production" story. Every senior engineer has one. The key is showing that you *owned it*, *fixed it fast*, and *changed the process so it never happened again*.

Part6_Scenarios_Set_1.md

DevSecOps & Cloud Security Architect Interview Guide: Scenarios Set 1

25 Advanced Real-World Attack Scenarios

1. **SSRF to IMDSv1 Metadata Theft**: An attacker uses a vulnerable web application to query `169.254.169.254` and steal EC2 instance IAM credentials.
2. **S3 Bucket Ransomware**: An attacker gains access to an S3 bucket, downloads the data, encrypts the objects in place using a KMS key they control, and deletes the originals.
3. **Lateral Movement via SSM**: An attacker compromises a developer laptop and uses AWS Systems Manager (SSM) Session Manager to seamlessly shell into private EC2 instances without needing SSH keys.
4. **Golden SAML Attack**: An attacker steals the ADFS token signing certificate and forges SAML tokens to bypass Azure AD / AWS SSO authentication entirely.
5. **Living off the Land (LOLBins)**: An attacker uses `certutil.exe` to download a malicious payload to bypass perimeter web filters.
6. **Docker Escape via Privileged Container**: An attacker compromises a pod running with `privileged: true`, mounts the host filesystem (`/dev/sda1`), and `chroot` s into the worker node.
7. **CloudTrail Evasion**: An attacker disables CloudTrail logging for a specific region, creates a backdoor IAM user, and then re-enables logging to hide their tracks.
8. **Kerberoasting**: An attacker requests service tickets for SPNs (Service Principal Names) and cracks the NTLM hashes offline to gain privileged Active Directory access.
9. **Log4Shell on EKS**: An attacker exploits CVE-2021-44228 on a Java-based microservice running in Kubernetes, gaining reverse shell access to the pod.
10. **Malicious Terraform Provider**: An attacker submits a PR that includes a compromised Terraform provider registry URL, executing malicious code during the CI/CD pipeline run.
11. **RDP Brute Force to Ransomware**: An attacker brute-forces an exposed RDP port, disables Windows Defender using `Set-MpPreference` , and deploys LockBit.
12. **Pass-the-Hash**: An attacker dumps LSASS memory using Mimikatz, extracts NTLM hashes, and authenticates to other domain machines without ever knowing the plaintext password.
13. **AWS GuardDuty Evasion**: An attacker uses an IP address previously whitelisted (e.g., a compromised corporate VPN IP) to perform reconnaissance, bypassing anomaly detections.
14. **Cross-Tenant AWS Abuse**: An attacker modifies an IAM Role Trust Policy to allow `sts:AssumeRole` from an AWS account number they control.

15. **Supply Chain Attack (NPM/PyPI)**: A developer accidentally installs a typosquatted Python package (`requessts` instead of `requests`) which opens a reverse shell during the Docker build process.
16. **DNS Data Exfiltration**: An attacker encodes stolen data into subdomains and queries a malicious DNS server to bypass outbound firewall restrictions.
17. **C2 via Domain Fronting**: An attacker hides Command and Control traffic behind a high-reputation CDN (like Cloudflare or CloudFront) to evade SIEM detection.
18. **Azure AD Illicit Consent Grant**: An attacker phishing email tricks a user into granting a malicious OAuth app permissions to read their O365 mailbox.
19. **Kubelet API Anonymous Access**: An attacker connects to an exposed Kubelet API on port 10250 and uses `/run` to execute commands directly on running pods.
20. **VPC Flow Log Blindness**: An attacker routes malicious traffic through AWS PrivateLink or VPC Peering to bypass traditional perimeter IDS/IPS appliances.
21. **Malicious Lambda Deployment**: An attacker updates an existing AWS Lambda function's code to silently forward all processed data to an external webhook.
22. **Container Registry Poisoning**: An attacker gains access to the company's ECR registry and replaces the `latest` tag of a core microservice with a backdoored image.
23. **BGP Hijacking**: (Conceptual) An attacker manipulates BGP routes to intercept traffic destined for the company's public IP space.
24. **Active Directory DCSync**: An attacker compromises a Domain Admin account and uses the Directory Replication Service (DRS) to pull all password hashes from the Domain Controller.
25. **Data Exfiltration via ICMP**: An attacker embeds stolen files within the data payload of ICMP Echo Request packets to bypass standard proxy monitoring.

---

20 True Positive (TP) vs False Positive (FP) Exercises

1. **Powershell.exe running with `-EncodedCommand`**. (Likely TP, requires decoding the Base64 to confirm. Often used by malware, but sometimes by SCCM).
2. **`whoami /all` executed by `cmd.exe`**. (Likely TP. This is classic reconnaissance. Standard users rarely run this).
3. **Nmap scanning activity originating from the Qualys scanner IP**. (FP. Authorized vulnerability scanning).
4. **`vssadmin.exe delete shadows /all /quiet`**. (TP. Absolute indicator of Ransomware preparing to encrypt).
5. **AWS CloudTrail showing `ConsoleLogin` without MFA**. (TP. Policy violation, unless it's a break-glass service account).
6. **CrowdStrike alerts on `psexec.exe`**. (Depends. If run by an IT admin for patching, FP. If run by an unknown user across 50 machines at 2 AM, TP).
7. **Impossible Travel: Login from New York and London within 10 minutes**. (Depends. If the London IP is a known corporate VPN or Zscaler node, FP. If it's a generic ISP, TP).
8. **Multiple failed SSH logins from a single IP, followed by a success**. (TP. Successful brute force attack).
9. **`rundll32.exe` communicating over the internet**. (TP. `rundll32` should generally not be making external network calls; often used to load malicious DLLs).
10. **High volume of `NXDOMAIN` DNS responses**. (TP. Indicator of malware using a Domain Generation Algorithm to find its C2).
11. **Developer executing `docker run --privileged` in development**. (FP from a threat perspective, but a policy violation from an architecture perspective).
12. **`svchost.exe` spawning `cmd.exe`**. (TP. `svchost` should not spawn command shells. Likely a hijacked service).
13. **AWS GuardDuty alerts on `UnauthorizedAccess:EC2/SSHBruteForce`**. (FP if it's the internet hitting the port, but the Security Group blocks it. TP if the Security Group allows it and the login succeeds).

14. **User downloads a ZIP file from an email, and `wscript.exe` executes a `.vbs` file inside it**. (TP. Classic phishing payload execution).
15. **Taegis XDR alerts on `mimikatz` string in memory**. (TP. Credential dumping).
16. **`aws s3 sync` command executed locally transferring 500GB of data**. (Depends. If it's the data engineering team, FP. If it's a compromised web server, TP/Exfiltration).
17. **A sudden spike in 500 Internal Server Errors on the WAF**. (TP. Likely an attacker fuzzing the application or attempting SQL injection).
18. **`schtasks.exe` creating a task named 'UpdateCheck' running from `%APPDATA%`**. (TP. Malware establishing persistence).
19. **Falcon alerts on a known malicious hash, but the action was 'Blocked'**. (TP that malware was present, but the incident is contained. Still requires investigation into *how* the hash arrived).
20. **AWS IAM `CreateAccessKey` called by a user who hasn't logged in for 90 days**. (TP. Likely a compromised dormant account).

---

20 EKS/Kubernetes Security Scenarios

1. **Unauthenticated Kube API**: The Kubernetes API is exposed to the internet `0.0.0.0/0` without requiring authentication.
2. **Default Service Account Abuse**: An attacker uses the automatically mounted service account token to query the API for secrets.
3. **Privileged Pod Breakout**: A pod deployed with `securityContext.privileged: true` allows an attacker to mount the underlying EC2 node's disk.
4. **Missing Network Policies**: An attacker compromises the frontend web pod and freely uses `curl` to reach the backend database pod because no network isolation exists.
5. **HostPath Mount Abuse**: A pod mounts `/var/run/docker.sock`, allowing an attacker to spin up new, completely uncontrolled containers on the host.
6. **Cleartext Secrets in etcd**: Kubernetes Secrets are not encrypted at rest using an AWS KMS key.
7. **Cluster-Admin Overprovisioning**: Developers are given `cluster-admin` RBAC roles instead of namespace-scoped access.
8. **EKS Node Group Vulnerabilities**: The underlying EC2 AMI for the EKS worker nodes is severely outdated and vulnerable to kernel exploits.
9. **Image Vulnerabilities (Log4j)**: A pod is deployed using an image with critical vulnerabilities because no Admission Controller (e.g., OPA Gatekeeper) blocks it.
10. **Egress Traffic Unrestricted**: A compromised pod initiates an outbound connection to a crypto-mining pool because there is no egress filtering.
11. **Helm Chart Poisoning**: A developer uses a publicly available, unverified Helm chart that contains a malicious sidecar container.
12. **Kube-proxy ARP Spoofing**: An attacker performs ARP spoofing inside the cluster network to intercept traffic between pods.
13. **Missing Pod Security Standards (PSS)**: Pods are allowed to run as root (`runAsNonRoot: false`).
14. **Dashboard Exposed**: The Kubernetes Dashboard is deployed publicly without authentication.
15. **Container Resource Exhaustion (DoS)**: A pod is deployed without CPU/Memory limits, and a malicious script causes it to consume 100% of the node's resources, starving other pods.
16. **Metadata Service Theft**: A pod accesses `169.254.169.254` to steal the worker node's IAM instance profile because IAM Roles for Service Accounts (IRSA) isn't used.
17. **Unauthorized Image Registries**: Pods are pulling images from Docker Hub instead of the approved internal Amazon ECR registry.
18. **Sidecar Injection Bypass**: An attacker modifies a deployment to remove the required security sidecar (e.g., a logging or proxy container).

19. **Compromised CI/CD Kubeconfig**: The Jenkins/GitLab runner's `kubeconfig` file is stolen, giving the attacker direct deployment access to the EKS cluster.
20. **eBPF Sensor Tampering**: An attacker with root privileges unloads the CrowdStrike Falcon eBPF sensor from the kernel, blinding the SOC to container activity.

Part7_Scenarios_Set_2.md

DevSecOps & Cloud Security Architect Interview Guide: Scenarios Set 2

20 AWS IAM Abuse Scenarios

1. `iam:CreateUser` **Abuse**: An attacker creates a new IAM user (`backup-admin`) for persistent backdoor access.
2. `iam:CreateAccessKey` **Abuse**: An attacker generates a second set of access keys for an existing admin user.
3. `iam:AttachUserPolicy` **Privilege Escalation**: An attacker with limited permissions attaches the `AdministratorAccess` managed policy to themselves.
4. `iam:UpdateAssumeRolePolicy` : An attacker modifies a role's trust policy to allow an external AWS account (the attacker's account) to assume it.
5. `sts:AssumeRole` **Chaining**: An attacker assumes a low-privilege role, which has permissions to assume a higher-privilege role, chaining them to reach Admin access.
6. `iam:PassRole` **to EC2**: An attacker creates an EC2 instance and passes an overly permissive `AdministratorAccess` role to it, then SSHes in to use the permissions.
7. `iam:PassRole` **to Lambda**: An attacker creates a Lambda function, passes it an Admin role, and sets the code to exfiltrate secrets or create users.
8. `iam:CreateLoginProfile` : An attacker sets a console password for an IAM user that previously only had API keys, allowing them GUI access.
9. **Inline Policy Injection (** `iam:PutUserPolicy` **)**: An attacker embeds a raw JSON policy directly onto a user to grant themselves `s3:*` permissions.
10. **Group Membership Manipulation (** `iam:AddUserToGroup` **)**: An attacker adds their compromised, low-privilege user into the `CloudAdmins` group.
11. **MFA Device Deletion (** `iam:DeactivateMFADevice` **)**: An attacker disables MFA on an admin account to maintain easier persistent access.
12. `iam:UpdateLoginProfile` : An attacker resets the console password of another legitimate user to hijack their session.
13. **CloudFormation Privilege Escalation**: An attacker with `cloudformation:CreateStack` uses it to deploy IAM roles they don't natively have permission to create.
14. **Cognito Identity Pool Abuse**: An attacker exploits an unauthenticated Cognito Identity Pool to obtain temporary AWS credentials with excessive permissions.
15. **S3 Bucket Policy Modification (** `s3:PutBucketPolicy` **)**: An attacker modifies a bucket policy to allow `Principal: "*"` to read sensitive data.
16. **KMS Key Deletion (** `kms:ScheduleKeyDeletion` **)**: A malicious insider schedules the deletion of the KMS key used to encrypt the company's main database, effectively destroying the data.
17. **CodeBuild Service Role Abuse**: An attacker modifies the `buildspec.yml` of an AWS CodeBuild project to exfiltrate the IAM role credentials assigned to the build runner.

18. **Systems Manager (SSM) Command Execution**: An attacker with `ssm:SendCommand` executes code as SYSTEM on all EC2 instances simultaneously without needing IAM keys on the instances themselves.
19. `iam:SetDefaultPolicyVersion` : An attacker reverts an IAM policy to an older, overly permissive version that the security team had previously fixed.
20. **IAM Role Session Name Spoofing**: An attacker assumes a role using `sts:AssumeRole` and sets the `RoleSessionName` to match a legitimate developer's email to throw off SOC investigations.

---

15 Ransomware Investigation Scenarios

1. **Patient Zero Identification**: Determining which endpoint was initially compromised 3 weeks before the encryption began.
2. **Double Extortion (Exfiltration before Encryption)**: Detecting `rclone` or `MegaSync` transferring 5TB of data to a cloud storage provider just prior to ransomware deployment.
3. **Lateral Movement via PsExec**: Investigating `psexec.exe` being executed across 50 servers from a single compromised Domain Controller.
4. **GPO-Based Ransomware Deployment**: Ransomware deployed via a malicious Active Directory Group Policy Object to instantly hit all domain-joined endpoints.
5. **VSS Deletion (Shadow Copies)**: Detecting `vssadmin.exe delete shadows /all /quiet` or `wmic shadowcopy delete` .
6. **Safe Mode Booting**: Ransomware configuring the machine to boot into Safe Mode (`bcdedit /set {default} safeboot minimal`) to bypass EDR software.
7. **RDP Brute Force Initial Access**: Investigating a server that had thousands of failed login attempts over port 3389 before a successful login and subsequent encryption.
8. **Malicious Macro Entry**: A user opens an Excel file, clicks "Enable Content", causing PowerShell to download Emotet, which eventually drops Ryuk.
9. **ESXi Hypervisor Ransomware**: Ransomware specifically targeting VMware ESXi servers and encrypting the underlying `.vmdk` files, bypassing Windows EDR.
10. **Service Deletion/Termination**: Ransomware systematically killing backup services (e.g., Veeam) and database services (e.g., MSSQL) before encryption to ensure files are unlocked.
11. **Defense Evasion (EDR Unhooking)**: Ransomware using API unhooking techniques to disable CrowdStrike Falcon's visibility before executing the payload.
12. **Living off the Land (BitLocker Abuse)**: Attackers using the native Windows BitLocker utility to encrypt drives and holding the recovery key for ransom, rather than using custom malware.
13. **Cloud Ransomware (S3 Versioning)**: Attackers encrypting an S3 bucket and deleting the previous versions because MFA Delete was not enabled.
14. **Time Delay Execution**: Ransomware scheduled to execute globally via Scheduled Tasks at 2:00 AM on a Sunday to maximize impact before the SOC can respond.
15. **The Ransom Note Drop**: Investigating the creation of `README.txt` files across thousands of directories using File Integrity Monitoring (FIM).

---

15 Phishing Investigation Scenarios

1. **Adversary-in-the-Middle (AiTM)**: A user clicks a link, is directed to an Evilginx2 proxy, and both their password and MFA session cookie are stolen.
2. **Malicious OAuth App Consent**: A user clicks "Accept" on a realistic-looking Microsoft login prompt, granting a malicious third-party app read access to their mailbox.
3. **Business Email Compromise (BEC)**: An attacker compromises the CEO's email and sends wire transfer instructions to the finance department.
4. **Right-to-Left Override (RTLO) Spoofing**: An attacker sends an executable attachment named `invoice_fdp.exe` but uses an RTLO character so it displays to the user as `invoice_exe.pdf` .

5. **Lookalike Domains (Typosquatting)**: Investigating an email originating from `microsofft.com` instead of `microsoft.com`.
6. **QR Code Phishing (Quishing)**: An email contains a QR code directing the user's mobile device to a credential harvesting site, bypassing corporate email URL scanners.
7. **HTML Smuggling**: A phishing email contains an HTML attachment. When opened, JavaScript inside the HTML dynamically generates a malicious `.zip` or `.iso` file directly in the browser, bypassing email attachment filters.
8. **SPF/DKIM/DMARC Failure**: An email is spoofed to look like an internal address, but checking the headers reveals it failed SPF and DMARC alignment.
9. **Malicious Inbox Rules**: After a successful phishing campaign, the attacker sets a rule to forward all emails containing the word "invoice" to an external address.
10. **Reply-Chain Phishing**: An attacker compromises a vendor's email account and replies to an existing, legitimate email thread with a malicious link, making it highly convincing.
11. **Credential Harvesting via SharePoint**: A legitimate compromised SharePoint account is used to host a document containing a link to a credential harvesting site.
12. **PDF with Embedded Link**: A PDF that contains no malware itself, but contains a clickable image that redirects to a phishing site.
13. **Password-Protected ZIPs**: An attacker sends a password-protected ZIP (with the password in the email body) to bypass antivirus scanning at the email gateway.
14. **Spearphishing targeting IT Admin**: A highly targeted email mimicking a Jira ticket alert sent to a Sysadmin to steal highly privileged credentials.
15. **Open Redirect Abuse**: A phishing link uses a legitimate corporate domain (e.g., `https://trusted.com/redirect?url=http://evil.com`) to bypass URL reputation filters.

---

20 Cloud Misconfiguration Scenarios

1. **Publicly Exposed S3 Bucket**: An S3 bucket containing PII has `Block Public Access` disabled and a bucket policy allowing `*`.
2. **Overly Permissive Security Groups**: Port 22 (SSH) and Port 3389 (RDP) open to `0.0.0.0/0` across multiple EC2 instances.
3. **Hardcoded AWS Credentials in GitHub**: A developer accidentally commits their `~/.aws/credentials` file to a public GitHub repository.
4. **Unencrypted EBS Volumes**: EC2 instances deployed without EBS encryption, risking data exposure if snapshots are shared publicly.
5. **No MFA for AWS Root User**: The root account has no virtual MFA device attached and is actively being used for administrative tasks.
6. **Public RDS Database**: An Amazon RDS instance is deployed in a public subnet with a security group allowing internet access.
7. **IAM Users with AdministratorAccess**: 50+ developers have the `AdministratorAccess` managed policy attached directly to their users instead of using groups or least-privilege roles.
8. **Unrestricted Egress Traffic**: A VPC has no outbound filtering (NAT Gateway open to `0.0.0.0/0`), allowing a compromised instance to easily download malware or exfiltrate data.
9. **CloudTrail Logging Disabled**: CloudTrail is not enabled for all regions, creating blind spots for API activity.
10. **IMDSv1 Enabled**: EC2 instances are running with IMDSv1 enabled, making them highly susceptible to SSRF-to-credential-theft attacks.
11. **Unsecured Lambda Environment Variables**: AWS Lambda functions containing raw API keys and database passwords in plaintext environment variables instead of Secrets Manager.
12. **Publicly Accessible EKS API Server**: The Amazon EKS control plane API endpoint is public and not restricted to corporate VPN IP ranges.

13. **Missing S3 Bucket Versioning/MFA Delete**: Critical data buckets lack versioning, making ransomware encryption permanent and irreversible.
14. **Broad KMS Key Policies**: A Customer Managed Key (CMK) has a key policy allowing any principal in the account to decrypt it.
15. **Dangling Elastic IPs (Subdomain Takeover)**: An Elastic IP is disassociated from an EC2 instance but still pointed to by a Route 53 DNS record, allowing an attacker to claim the IP and serve malicious content on the company's subdomain.
16. **SNS Topic Public Access**: An Amazon SNS topic policy allows `Publish` from any AWS account, enabling spam or malicious payload injection.
17. **ECR Image Tag Mutability**: ECR repositories are set to mutable, allowing an attacker to overwrite a legitimate `latest` image with a backdoored version.
18. **Unrestricted IAM PassRole**: A developer role has `iam:PassRole` for `Resource: *`, allowing them to pass Administrator roles to EC2 instances they create.
19. **Default VPC in Use**: Production workloads are deployed in the Default VPC with default security groups instead of a custom, segmented network architecture.
20. **No GuardDuty/Security Hub Enabled**: Core AWS threat detection services are completely disabled, leaving the SOC blind to cloud-native attacks.

Part8_Scenarios_Set_3_and_Reporting.md

DevSecOps & Cloud Security Architect Interview Guide: Scenarios Set 3

15 Detection Tuning Exercises

1. **Rule**: Alert on any use of `whoami`. **Problem**: Triggers 500 times a day because the SCCM client uses it during software deployment. **Tuning Solution**: Exclude the specific Parent Process (`ccmexec.exe`) running from the exact System Center installation directory.

2. **Rule**: Alert on AWS `ConsoleLogin` without MFA. **Problem**: Triggers constantly for a specific service account that doesn't support virtual MFA. **Tuning Solution**: Create an exception for that specific IAM User ARN, but enforce a secondary compensating control rule (e.g., alert if that user logs in from any IP other than the corporate NAT Gateway).

3. **Rule**: Alert on high volume of HTTP 403 Forbidden errors (brute force detection). **Problem**: An outdated mobile app version is hitting a deprecated API endpoint, causing 10,000 FPs a day. **Tuning Solution**: Filter out the User-Agent specific to that old mobile app version, while retaining the rule for all other traffic.

4. **Rule**: Alert on PowerShell execution with `-ep bypass`. **Problem**: The DevOps team uses this in a Jenkins build script globally. **Tuning Solution**: Whitelist the specific hash of the Jenkins build script, or restrict the exclusion to the Jenkins worker node hostnames/IPs.

5. **Rule**: Alert on outbound RDP (3389). **Problem**: The IT Helpdesk uses RDP daily for remote support. **Tuning Solution**: Create an Active Directory Group exclusion (`Helpdesk_Admins`) so the alert only fires if a *non-IT* user initiates an RDP session.

6. **Rule**: Alert on `aws s3 sync` execution. **Problem**: Data engineers use this command hourly to back up logs. **Tuning Solution**: Exclude the specific IAM Role (`DataEngineeringRole`) used by the automated pipeline, but keep the alert active for all human IAM users.

7. **Rule**: Alert on `curl` or `wget` execution on Linux servers. **Problem**: Triggers during automated package updates (`apt-get` / `yum` post-install scripts). **Tuning Solution**: Exclude `curl` / `wget` when the parent process is the package manager (`dpkg` or `rpm`), or when the destination URL is an official Ubuntu/CentOS repository.
8. **Rule**: Alert on massive file deletion (Ransomware/Wiper detection). **Problem**: A log rotation script naturally deletes thousands of old `.log` files every night at midnight. **Tuning Solution**: Whitelist the specific script path and bound the exclusion to the scheduled 12:00 AM - 12:05 AM time window.
9. **Rule**: Alert on impossible travel (e.g., login from US, then India 5 mins later). **Problem**: The CEO travels and uses a commercial VPN on their phone. **Tuning Solution**: Integrate the SIEM with Azure AD to recognize 'Known Good Devices' or whitelist known commercial VPN ASN ranges for executives, requiring a secondary risk factor (like a new device) to trigger.
10. **Rule**: Alert on `net user /add` . **Problem**: The desktop provisioning script creates local admin accounts on first boot. **Tuning Solution**: Exclude the alert if it occurs within 10 minutes of the system's first boot/uptime timestamp, and only if spawned by the deployment service.
11. **Rule**: Alert on base64 encoded PowerShell commands. **Problem**: Microsoft Exchange Server naturally generates massive amounts of base64 PowerShell during normal operation. **Tuning Solution**: Create a strict exclusion for Exchange Servers (`Parent Process: w3wp.exe` originating from the Exchange install path).
12. **Rule**: Alert on new EC2 instance creation. **Problem**: The Auto Scaling Group scales up and down constantly. **Tuning Solution**: Exclude `RunInstances` API calls made by the `AWSServiceRoleForAutoScaling` role.
13. **Rule**: Alert on suspicious child processes of Microsoft Word (`winword.exe`). **Problem**: A legacy financial plugin genuinely spawns `cmd.exe` to check a local license file. **Tuning Solution**: Do not whitelist `cmd.exe` entirely! Whitelist the exact command line string (e.g., `cmd.exe /c type C:\license.txt`) so other malicious commands still trigger.
14. **Rule**: Alert on any AWS Security Group change. **Problem**: Terraform pipelines destroy and recreate security groups daily during testing. **Tuning Solution**: Exclude the Terraform Jenkins execution role, but monitor if the SG change opens ports `22` or `3389` to `0.0.0.0/0` (a "never allow" condition regardless of the user).
15. **Rule**: Alert on `vssadmin.exe` execution (Shadow copy deletion). **Problem**: A third-party backup agent uses `vssadmin.exe` to manage snapshots. **Tuning Solution**: Whitelist the code-signing certificate of the legitimate backup vendor, ensuring that if malware renames itself to the backup agent, it still triggers because the signature will be invalid.

---

15 SOC Manager-Level Reporting Questions

Q1: The CISO asks for a weekly report on SOC performance. What 5 metrics do you include and why?

Answer:

1. **MTTD (Mean Time to Detect):** How fast we spot the bad guys.
2. **MTTR (Mean Time to Respond):** How fast we contain them.
3. **True Positive Ratio (Fidelity Rate):** Are our rules noisy, or are they accurate?
4. **Alerts per Analyst / Burnout Rate:** To ensure we aren't overwhelming the team.
5. **Coverage Gaps (e.g., % of endpoints missing Falcon):** To show risk outside the SOC's immediate control.

Q2: How do you justify the budget for a SOAR platform to the Board of Directors?

Answer: "A SOAR platform is an ROI multiplier. Currently, our Level 1 analysts spend 20 minutes manually triaging a single phishing email. We receive 500 a week. That's 166 hours of manual labor. A SOAR platform automates this triage, reducing the time to 1 minute per email. This allows us to reallocate 3 full-time analysts from repetitive copy-pasting to proactive threat hunting and cloud architecture, drastically lowering our breach risk without adding headcount."

Q3: A major zero-day vulnerability (like Log4Shell) drops on a Friday night. Walk me through your communication and execution plan as the SOC Lead.

Answer:

1. **Declare an Incident:** Open a priority bridge.
2. **Triage:** Query the SIEM/Falcon to see if we have active exploitation attempts against our perimeter.
3. **Identify:** Pull a Nessus or CrowdStrike Spotlight report to identify all vulnerable assets.
4. **Communicate:** Send an initial brief to the CISO: "We are aware of CVE-X. We have Y vulnerable assets. We are seeing Z exploit attempts but no successful breaches. We are deploying WAF blocking rules now."
5. **Remediate:** Coordinate with IT to patch internet-facing assets immediately.

Q4: You notice MTTR is steadily increasing over the last 3 months. How do you investigate the root cause?

Answer: I look at three areas: People, Process, and Technology.

- *People*: Have we lost senior analysts, leaving juniors to handle complex alerts?
- *Process*: Are the playbooks outdated, requiring analysts to guess what to do?
- *Technology*: Is the SIEM searching slowly? Did we turn on a new log source that flooded the queue?

Q5: How do you build a Detection Engineering lifecycle?

Answer: It's a continuous loop:

1. **Threat Intel:** Read about a new attack (e.g., APT29 using a new technique).
2. **Hypothesis:** Assume we are compromised by it.
3. **Hunt:** Search the SIEM for the behavior.
4. **Code:** Write the detection rule.
5. **Test:** Execute a red-team simulation (e.g., using Atomic Red Team) to ensure the rule fires.
6. **Tune:** Reduce false positives.
7. **Deploy:** Push to production.

(Remaining 10 questions focus on strategic thinking)

6. **How do you measure the effectiveness of your Threat Intelligence feeds?** (Look at hit rates. If a feed costs $50k/year but hasn't generated a single True Positive alert in 6 months, it's low value).
7. **What is the difference between an SLA (Service Level Agreement) and an SLO (Service Level Objective) in the SOC?** (SLA is a contractual obligation, often with penalties; SLO is an internal goal for MTTR/MTTD).
8. **How do you handle 'Alert Fatigue' among your analysts?** (Aggressive rule tuning, SOAR automation, and rotating analysts out of the queue into project work/hunting).
9. **How do you map SOC coverage to the MITRE ATT&CK framework for executive reporting?** (Use a heat map showing which techniques we have strong detections for vs. blind spots).
10. **A penetration test report comes back with a "Critical" finding that the SOC completely missed. How do you respond?** (Do a blameless post-mortem. Why didn't it fire? Was it a lack of logs, a broken rule, or

analyst error? Fix the gap).

11. **How do you integrate the SOC with the DevOps/Engineering teams?** (Create a DevSecOps culture—embed security champions in the dev teams, and ensure SOC alerts have clear, actionable remediation steps for engineers).
12. **What is your strategy for retaining top SOC talent?** (Pay for certifications, allow them time for research/hunting, and automate the boring L1 work so they can focus on complex analysis).
13. **How do you report Cloud Security posture (AWS) to non-technical leadership?** (Use simple metrics: "Percentage of public S3 buckets," "Number of overly permissive IAM roles," and trend lines showing improvement over time).
14. **When do you decide to escalate a security event to a full-blown Critical Incident?** (When there is confirmed unauthorized access to sensitive data, widespread lateral movement, or an active ransomware deployment).
15. **How do you ensure your SOC playbooks remain relevant?** (Schedule quarterly reviews, and mandate that every post-incident report includes a section on "Playbook Updates Required").

Part9_Final_Evaluation_and_Roadmap.md

DevSecOps & Cloud Security Architect Interview Guide: Final Evaluation

My Weak Areas Based on Resume

1. **Length of Experience vs. Senior Titles:** You have 4 years of experience. Applying for "Senior Architect" or "SOC Manager" roles might raise eyebrows. You need to compensate for the *duration* by emphasizing the *depth* and *complexity* of what you've handled (e.g., EKS DaemonSets, CWPP).
2. **Heavy Tool Focus over Conceptual Depth:** Your resume lists many tools (Falcon, Taegis, Wazuh, Splunk). Interviewers might suspect you only know how to click buttons in a UI. You must prove you understand *how* the tools work under the hood (e.g., eBPF in Falcon, API queries in AWS).
3. **Architecture/Design Experience:** A SOC Analyst role is highly reactive. An Architect role is proactive. Your resume is very strong on response/hunting but lighter on initial network/cloud design from scratch.
4. **DevSecOps Depth:** You mention Terraform and Docker/Kubernetes, but "basic" next to them in your skills list is a red flag for senior roles. You need to remove the word "basic" and speak confidently about integrating security into pipelines.

What Interviewers Are Likely to Challenge Me On

1. **"You claim you managed Falcon CWPP on EKS. Walk me through the exact deployment architecture. How did you handle RBAC for the sensor?"** (They are checking if you actually deployed it or just monitored the dashboard).
2. **"You mention MITRE ATT&CK. Tell me exactly how you mapped a specific threat hunt to a MITRE Tactic and Technique, and what the resulting detection looked like."** (Checking if it's just a buzzword).
3. **"How do you distinguish between a False Positive and True Positive for an AWS IAM abuse alert?"** (Testing your analytical methodology and AWS knowledge).
4. **"If I give you a blank AWS account, how would you design the security architecture from the ground up?"** (Testing your transition from Analyst to Architect).

What Topics I Should Study Deeper

1. **AWS Identity and Access Management (IAM):** Understand `sts:AssumeRole`, Instance Profiles, Cross-Account access, and SCPs natively. This is the #1 attack vector in the cloud.
2. **Kubernetes Architecture:** Understand the difference between the Control Plane (API Server, etcd) and the Data Plane (Kubelet, worker nodes). Understand how Admission Controllers (OPA Gatekeeper) and Network Policies work.
3. **DevSecOps Pipelines:** Be able to draw on a whiteboard how code moves from a developer's laptop -> Git -> CI/CD Runner (Jenkins/GitLab) -> Docker Registry (ECR) -> EKS, and where exactly security tools (SAST, SCA, DAST, Image Scanning) fit into that flow.
4. **Server-Side Request Forgery (SSRF) & IMDS:** Deeply understand how web application vulnerabilities lead to cloud infrastructure compromise.

Final 7-Day Preparation Roadmap

- **Day 1: Resume Mastery & Narrative.** Re-read your resume. Prepare a STAR (Situation, Task, Action, Result) story for *every single bullet point*. Never be caught off-guard by your own resume.
- **Day 2: AWS Deep Dive.** Review AWS IAM privilege escalation paths. Memorize how to investigate CloudTrail logs for `ConsoleLogin`, `AssumeRole`, and `CreateAccessKey`.
- **Day 3: Kubernetes & Falcon.** Review the CrowdStrike documentation for deploying on Kubernetes. Understand DaemonSets, `hostPID`, and kernel monitoring.
- **Day 4: Incident Response.** Practice walking through the SANS IR lifecycle for three scenarios: Phishing, Ransomware, and AWS IAM credential theft. Speak out loud.
- **Day 5: DevSecOps & Architecture.** Map out a CI/CD pipeline on paper. Know the difference between SAST, DAST, and SCA. Be ready to explain "Shift-Left".
- **Day 6: Mock Interview (Out Loud).** Record yourself answering the questions from *Part 1* and *Part 2* of this guide. Listen to the playback. Are you saying "um"? Are you rambling? Keep answers under 3 minutes.
- **Day 7: Rest and Mindset.** Do not study new material. Review your top 3 success stories. Focus on your breathing and confidence.

A Confidence-Building Strategy for Interviews

1. **The "Consultant" Mindset:** Do not go into the interview thinking "Please hire me." Go in thinking, "I am a security consultant evaluating if my skills can solve their current problems." This shifts the power dynamic and relaxes you.
2. **You Know More Than You Think:** The interviewer likely doesn't know everything you know. They might be an expert in AppSec but know very little about CrowdStrike EDR. Don't assume they are trying to trick you; often, they are just curious about how *you* solved a problem.
3. **The Power of "I Don't Know, But...":** If you get a question you don't know, never panic or lie. Say: "*I haven't encountered that specific scenario in my environment. However, based on my understanding of X, my approach to investigating it would be Y.*" This shows analytical thinking, which is more valuable than memorization.
4. **Control the Pace:** When asked a complex architecture question, say: "*That's a great question. Let me take 10 seconds to structure my thoughts.*" Take a sip of water, outline your 3 main points in your head, and then answer. It projects immense seniority and control.
5. **Remember Your Wins:** Before you log into the Zoom call, remind yourself: *You have 4 years of experience. You have secured production Kubernetes clusters. You have hunted real threats. You belong in this room.*
